LAB MANUAL: Harris Corner Detection

Objective

- Understand corner detection
- Apply Harris Corner Detection
- Visualize detected corners
- Understand mathematical working of Harris detector

Theory

Harris Corner Detection identifies points where intensity changes significantly in both x and y directions.

Corner Response Function:
$R = \det(M) - k(\text{trace}(M))^2$

Where M is structure tensor matrix.

If R is large positive → Corner
If R is small → Flat region
If R is negative → Edge

Step 1: Import Libraries

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Working:
cv2 → Image processing
numpy → Mathematical operations
matplotlib → Display images

Step 2: Read Image

Code:

```
img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)
```

Working:

Image is converted to grayscale.
Harris works on single channel images.
Converted to float32 because algorithm requires float type.

Step 3: Apply Harris Corner Detection

Code:
```
dst = cv2.cornerHarris(gray, 2, 3, 0.04)
```

Working:
2 → Block size (neighborhood size)
3 → Sobel kernel size
0.04 → Harris detector free parameter k
Algorithm computes corner response R for each pixel.

Step 4: Dilate Result

Code:
```
dst = cv2.dilate(dst, None)
```

Working:
Dilates corner points.
Makes corners more visible for marking.

Step 5: Mark Corners on Image

Code:
```
img[dst > 0.01 * dst.max()] = [0, 0, 255]
```

Working:
Threshold is applied.
Pixels with strong corner response are marked red.
Final image shows detected corners.

Step 6: Display Result

Code:
```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Harris Corners")
plt.show()
```

Working:

Convert BGR to RGB for correct display.
Final image displays detected corners.

Conclusion

Harris Corner Detection finds points where intensity changes in both horizontal and vertical directions.

Applications:
- Feature detection
- Object tracking
- Image matching
- Crack intersection detection
- Computer vision systems